

응용 · GEMINI CLI · VS CODE 강의 교안

Coding Agent 실전

AI기반 TDD 마스터 과정

개념에서 코드까지, 추적성을 잃지 않는 개발 (Concept-to-Code)

4계층 검증 구조

계약 ID

추적성 매트릭스

Dual-Track TDD $\equiv$ ARRR

하네스 엔지니어링

하루 8시간 집중 과정 · 08:30-17:30 | 강의 40% + 실습 60% | 개발자 및 입문자 대상

2026 · 김대경 Ph.D.

과정 개요 — 하루 만에 C2C 개발 흐름을 체득한다

항목	내용
교육 대상	개발자 및 입문자 (Git 기초 숙지 전제)
교육 시간	8시간 — 08:30~17:30 (점심 11:45~13:00) · 0장(사전 과제)은 시간에서 제외
교육 방식	강의 40% + 실습 60% (개념 → 시연 → 개별 실습)
도구 스택	VS Code · Gemini CLI(터미널 AI 에이전트) · Gemini Code Assist
사전 과제	0장(설치·CLI 기본기)은 교육 전 개별 완료 — 당일엔 체크리스트만 확인
관통 예제	장바구니 할인 계산기 — 계약 ID 6개(INV·E·AC·U)를 ARRR로 완성

핵심 목표 — 개념이 코드까지 추적되는(C2C) 개발 흐름을 1일 안에 체득한다. 산출물: docs/PRD.md · tests/ · src/ · docs/traceability.md

00

환경 준비 — 설치·CLI 기본기는 미리 끝낸다

Chapter 0 · 사전 과제 (강의 시간에서 제외). 0장은 교육 전 개별 완료하고, 당일 0교시에는 체크리스트만 5분 확인한 뒤 바로 이론으로 진입한다.

준비물 점검 — 시작 전에 갖춰야 할 것

개인 PC

Windows · macOS · Linux (듀얼 모니터 권장)

GitHub · Google 계정

무료 등급으로 CLI·Code Assist 로그인

Node.js 20+ · Python 3.11+ · Git

CLI 설치와 pytest·Flask 실습에 사용

환경 — 버전 확인

```
# 이 네 줄이 통과해야 시작
$ node --version # v20.x 이상
$ python --version # 3.11.x 이상
$ git --version # 2.x
$ gemini --version # 없으면 0.2에서 설치
```

⚠ 가장 흔한 사전 준비 실패 원인 — Node.js가 20 미만이면 Gemini CLI 설치가 실패한다. 반드시 먼저 확인한다.

도구 스택과 설치 — 세 가지의 역할 분담

VS Code

에디터 + 통합 터미널

코드 열람·수동 편집, 터미널에서
gemini·pytest 실행

Gemini CLI

터미널 AI 에이전트

다중 파일 편집·테스트 자동 실행·자율 반복
(ARRR 주력)

Gemini Code Assist

VS Code 확장(채팅·Agent)

코드 질의·인라인 편집·Agent 탭 (CLI와 동일
엔진)

Gemini CLI 설치 명령

```
# gemini-cli 설치  
npm install -g @google/gemini-cli
```

💡 수업 시 Gemini Code Assist는 사용하지 않는다. CLI 중심으로 진행한다.

Gemini CLI 기본기 — 대화 시작 방법 5가지

대화 시작 방법	무엇·언제	예시 (gemini-at-practice-python)
gemini	여러 단계의 대화형 개발	gemini
gemini -p "..."	한 번 분석하고 종료	gemini -p "@src/utils/math_utils.py 설명해줘"
@경로	코드/문서를 명시적 컨텍스트로 지정	@src/utils/math_utils.py 경계조건을 찾아줘
!명령	실제 프로젝트 명령 실행	!python -m src.main, !pytest -q
/명령	Gemini CLI 기능 실행	/help, /model, /mcp

승인 모드 — plan(읽기 전용) · default(매번 승인, 권장 시작점) · auto_edit(편집만 자동) · yolo(전부 자동, 격리 저장소에서만).
△ 처음엔 반드시 default로 시작한다.

01

Dual-Track TDD — RED → GREEN 으로 추적 가능한 개발

1부 · 왜 테스트를 먼저 두는가. Gemini CLI · Python 3.12 · pytest · Git | 하루 8시간 집중 과정

왜 “테스트 먼저”인가

판단의 조급성 · Discovery

문서는 바람(needs)의 나열이었을 뿐, 서로가 합의하고 증거로 판정할 수 있는 가설·수용조건이 아니다.

[시스템 '개발' 현장의 모습]



행동의 조급성 · Delivery

검증 근거 없이 구현이 앞서면, 통과한 것처럼 보이지만 실제로는 아닌 코드가 쌓인다.

[시스템 '개발' 운영의 모습]



판단이 곧고 개발(행동)이 끝나갈 무렵에야 “이게 요구사항이었나?”라는 질문이 테스트 시점에 뒤늦게 터진다. 질문이 판단 행동을 앞서지 못하고 맨 뒤로 밀려난 전형이다.

빠르게만 만들면 생기는 문제들



검증 근거 부재

이 코드가 정말 요구사항을 만족하는지 객관적으로 보여 줄 근거가 없다



추적 어려움

어떤 변경이 무엇을 왜 고쳤는지 나중에 따라가기 어렵다



위장된 통과

세션이 길어질수록 통과한 것처럼 보이지만 실제로는 아닌 코드가 는다

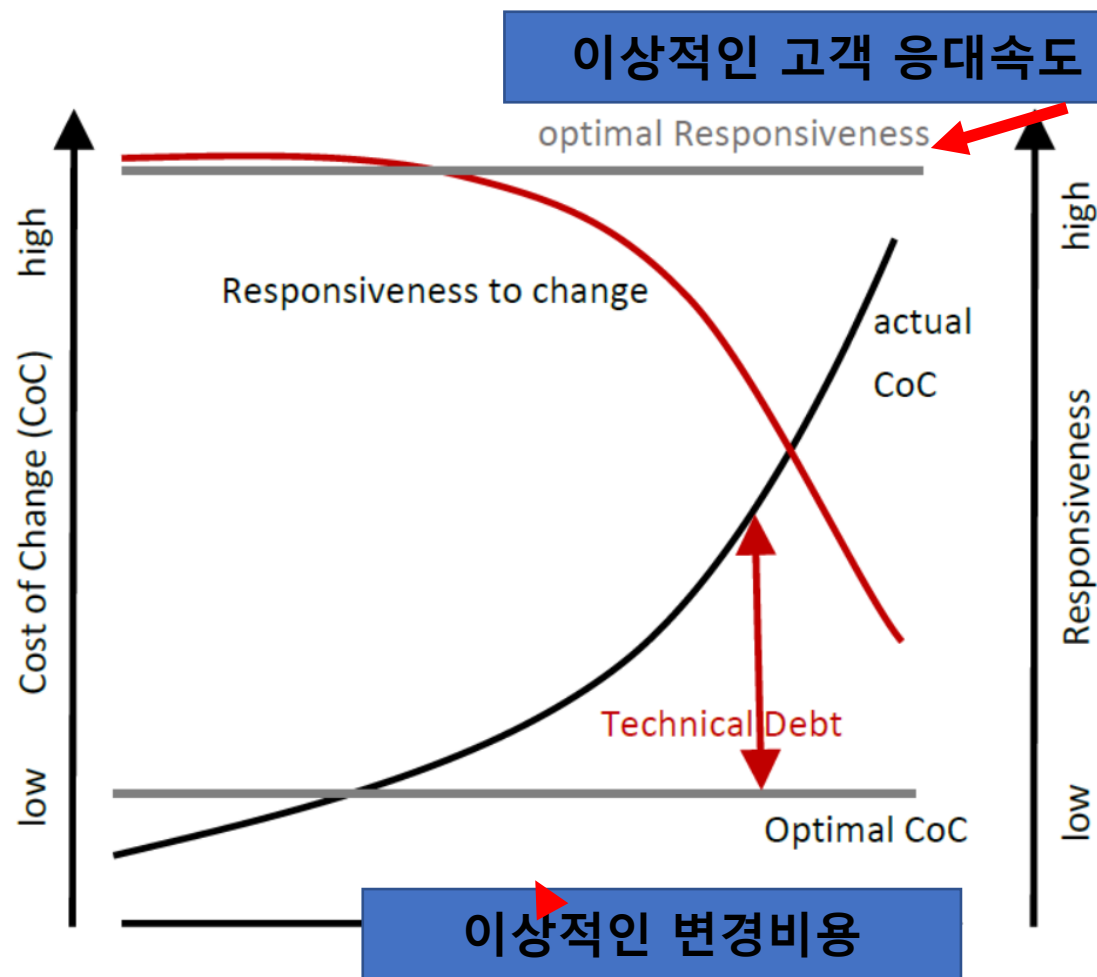
전통적 방식

기획서 → 개발자 해석 → 구현 → (뒤늦게) 문제 발견

Dual-Track TDD

실패 조건 합의 → RED 테스트 → 최소 구현(GREEN)

클린코드와 변경 비용 — 시간이 지날수록



시간이 지날수록...

변경 비용

클린코드 ✕ → 기하급수적 증가

클린코드 ✓ → 완만하게 유지

응답 속도

클린코드 ✕ → 점점 느려짐

클린코드 ✓ → 빠른 응대 유지

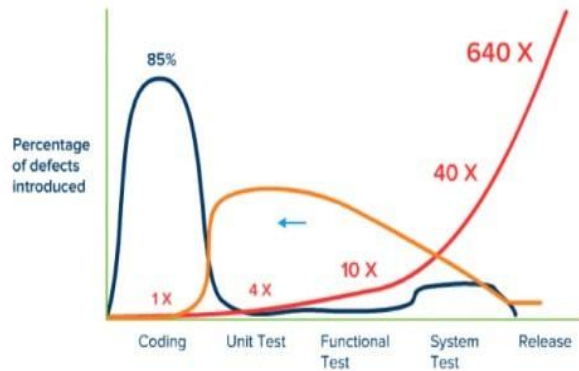
버그 발생

클린코드 ✕ → 수정 시 부작용

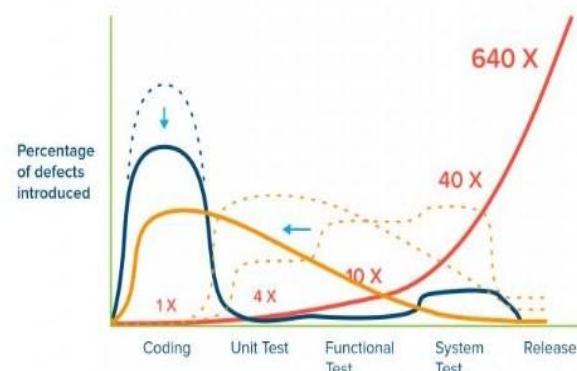
클린코드 ✓ → 테스트로 즉시 감지

테스팅의 비용 — 결함은 늦게 발견할수록 비싸다

결함은 늦게 발견할수록 수정 비용이 기하급수적으로 증가한다 (Cost of Defect).



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



Jones, Capers. *Applied Software Measurement: Global Analysis of Productivity and Quality*.



💡 TDD: 개발 시간 15~35% 증가 → 결함률 40~90% 감소 (Microsoft·IBM 연구)

Testing의 4영역과 두 가지 태도

애자일 테스트 사분면(Q1~Q4)은 테스트의 목적을 “기술/비즈니스”와 “팀 지원/제품 비판” 축으로 나눈다.



다음 중 개발팀을 지원하는 테스트는?

1. Q1, Q2
2. Q3, Q4
3. Q1, Q4
4. Q2, Q3

변경 후 기도하기 (Edit & Pray)

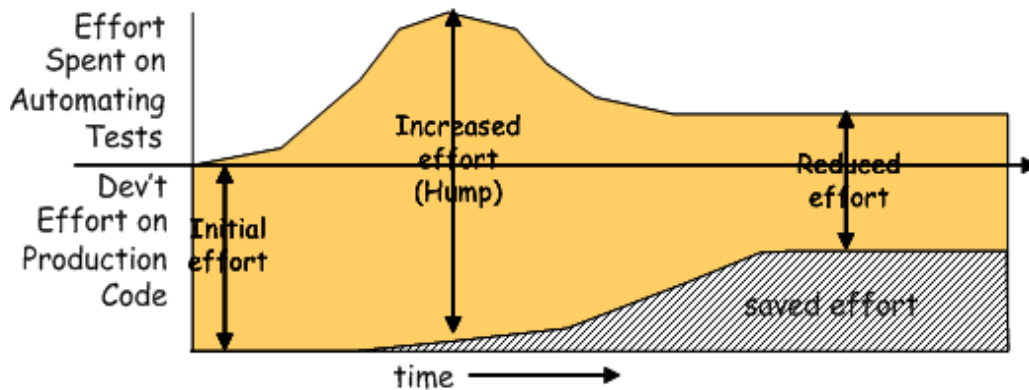
체계적 보호 없이 변경 → 위험이 높다

보호 후 수정하기 (Cover & Modify)

테스트로 코드를 보호 → 문제를 즉시 감지

테스트 자동화 — “올바르게” 자동화하는 것이 핵심

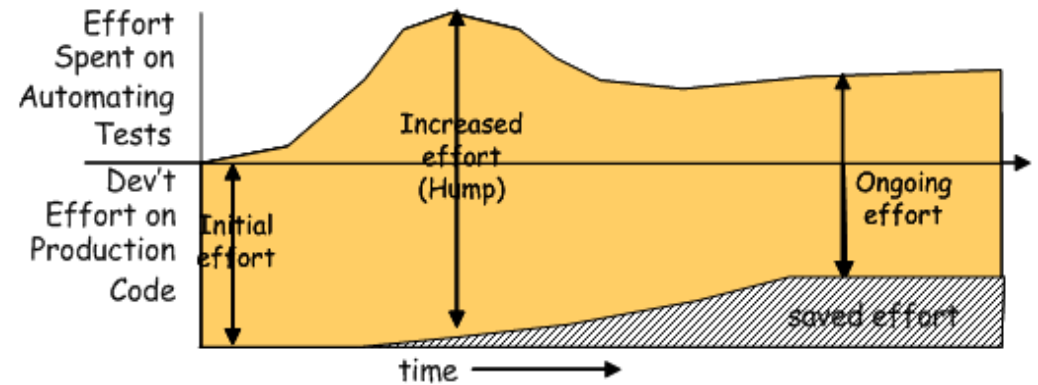
단순히 자동화 테스트를 도입하는 것이 중요한 것이 아니라, “올바르게 자동화하는 것”이 ROI를 높이는 핵심이다.



Sketch Economics-Good embedded from Economics-Good.gif

Fig. X: An automated unit test project with a good ROI.

The cost-benefit trade off where the total cost is reduced by good test practices.



Sketch Economics-Bad embedded from Economics-Bad.gif

Fig. X: An automated unit test project with a poor ROI.

The cost-benefit trade off where the total cost is increased by poor test practices.

반복 가능성

같은 명령으로 언제나 실행되는 검증이 “객관적 진실”이 된다

빠른 피드백

실패를 즉시 발견해 원인 추적 비용을 줄인다

회귀 방지

기존 동작이 깨졌는지 매 변경마다 자동으로 확인한다

단위 테스트가 중요한 이유 — F.I.R.S.T 원칙 (Python)

테스트가 없는 코드를 수정하면?
내가 고친 코드가 어떤 영향을 끼칠지 몰라 수정이 어렵다. 결국 코드는 썩는다.

테스트가 잘 갖춰진 코드를 수정하면?
잘못 고쳐도 테스트가 잡아 주니 편하게 수정한다. 리팩토링 자유도가 높아진다.

원칙	설명	Python (pytest)
Fast	테스트는 빨리 돌아야 한다	함수 단위 — ms 단위 실행
Independent	테스트는 서로 의존하면 안 된다	fixture로 초기화 (@pytest.fixture)
Repeatable	어떤 환경에서도 같은 결과	@pytest.mark.parametrize
Self-Validating	결과는 bool로 자동 판단	assert expected == actual
Timely	테스트는 실 코드 직전에 작성	TDD — 테스트 함수 먼저 작성

Right-BICEP & CORRECT — 무엇을 테스트할 것인가 (Python)

Right-BICEP

R — Right	결과가 옳은가? <code>assert add(5,5)==10</code>
B — Boundary	경계조건? <code>sys.maxsize, 0, None</code>
I — Inverse	역관계? <code>sorted</code> →역순 후 원본
C — Cross-check	다른 수단으로 교차 검증?
E — Error	에러 조건? <code>pytest.raises</code>
P — Performance	성능 조건? 타임아웃 측정

CORRECT (경계 조건)

C — Conformance	형식 준수: 이메일·날짜·JSON
O — Ordering	정렬·순서: 오름차순 보장
R — Range	범위: 나이 0~150, 인덱스
R — Reference	참조 유효성: 없는 ID·삭제된 객체
E — Existence	존재: None·빈 문자열·0
C — Cardinality	개수: 최소 1, 최대 100
T — Time	시간: 만료·타임아웃·동기화

AAA 패턴 — 좋은 테스트의 구조 (Python)

모든 테스트는 Arrange(준비) → Act(실행) → Assert(검증) 3단계로 명확히 구분한다.

Python — pytest

```
def test_mean_score_is_calculated():  
    # Arrange — 준비  
    collection = ScoreCollection()  
    collection.add(5)  
    collection.add(7)  
  
    # Act — 실행  
    result = collection.arithmetic_mean()  
  
    # Assert — 검증  
    assert result == 6
```

AAA 단계	pytest	핵심 원칙
Arrange	fixture / 객체 생성	테스트 픽스처 분리
Act	메서드 호출 1줄	단 하나의 동작
Assert	assert 문	명확한 성공/실패

테스트 케이스란 무엇인가

테스트 케이스(Test Case)란 요구사항을 준수하는지 검증하기 위하여 테스트 조건, 입력값, 예상 출력값 및 수행 결과 등 테스트 조건을 명시한 것이다.

테스트 케이스 = 특정 조건 + 입력값 + 기대 결과

실행 사전 조건

테스트를 수행하기 위한 전제 상태

입력값의 묶음

특정 프로그램 경로를 실행하도록 설계된 입력
집합

기대 결과·사후 조건

요구사항과 일치하는지 판정할 예상 출력

테스트 케이스 작성 절차 — 사칙연산 예 (Python)

테스트단계	입력값	예상값	중요도	성공/실패
1 + 10	1, 10	11	상	성공
0 + 1	0, 1	1	상	성공
0 ÷ 0	0, 0	ZeroDivisionError 발생	상	성공
-1 + (-10)	-1, -10	-11	중	성공

테스트 환경

Python 3.11+

pytest 8.x

VS Code + 통합 터미널

운영체제: Windows / macOS / Linux

```
import pytest
from src.utils.math_utils import add, divide

def test_add_positive():    assert add(1, 10) == 11
def test_divide_by_zero():
    with pytest.raises(ZeroDivisionError):
        divide(0, 0)
```

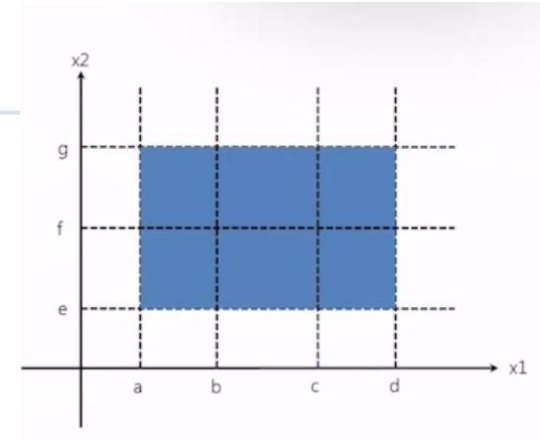
02

블랙박스 테스트 — 명세 기반 테스트 케이스 설계

2부 · 동등 분할 · 페어와이즈 · 경계값 분석 · 시나리오 테스트. 내부 구조를 보지 않고, 입력·출력 명세만으로 테스트를 설계한다.

동등 분할 (Equivalence Partitioning)

- 기본 아이디어 — 입력 데이터를 동일한 동작이 예상되는 클래스로 분할한다
- 각 클래스의 대표값 하나를 선택하여 테스트 데이터를 설계한다
- 분할된 클래스에 속한 값은 모두 같다고 가정한다



Example — 성적 판정 (0~100)

0~59 → F (대표값 30)

60~69 → D (대표값 65)

70~79 → C (대표값 75)

80~89 → B (대표값 85)

90~100 → A (대표값 95)

이점

- 누락 없이 완전하게 테스트할 수 있다
- 중복 없이 테스트할 수 있다
- 무게 판정·택배 요금 등 구간이 있는 모든 입력에 적용된다

동등 분할의 변형 — Normal / Robust

Weak Normal

Single fault 가정 — 한 변수의 동치 클래스 구간만 이용, 가장 많은 변수 선택

Strong Normal

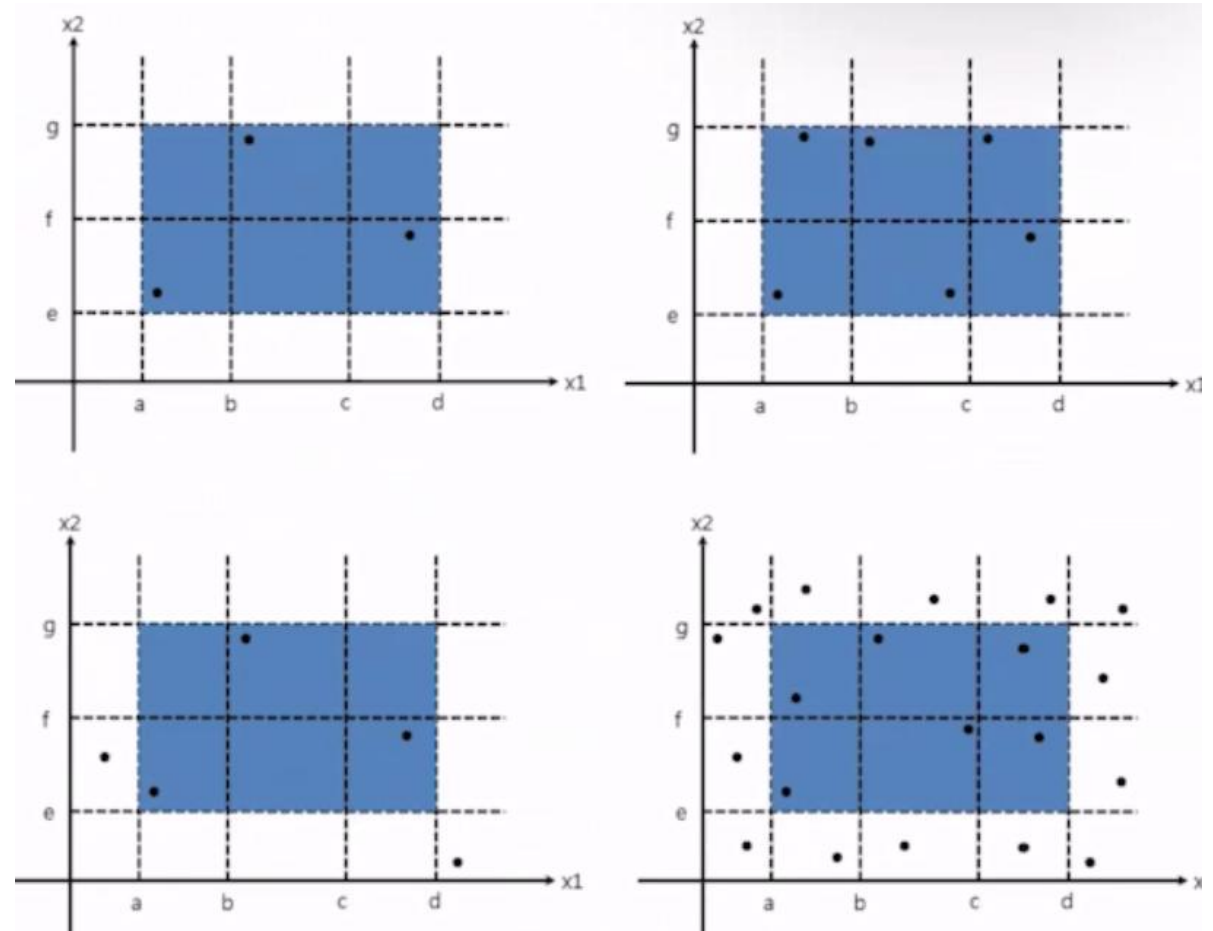
Multiple fault 가정 — 모든 변수 간의 곱집합(조합)을 이용

Weak Robust

Weak Normal + 범위 밖(비정상) 값 추가

Strong Robust

Strong Normal + 범위 밖(비정상) 값 추가



페어와이즈(Pairwise) 테스트

모든 가능한 입력값 조합을 테스트하는 대신, 모든 짝(pair)의 조합만 테스트한다. 조합 폭발을 막으면서 결함 검출력은 높게 유지한다.

Example — 피자 주문 (입력 인자)

- 도우: 곡물 · 나폴리 · 썬 크리스트 (3)
- 토핑: 기본 · 프리미엄 (2)
- 주문: 전화 · 온라인 (2)

All combinations = $3 \times 2 \times 2 = 12$ 케이스

Pairwise = 6 케이스로 축소 (모든 짝 커버)

언제 쓰나

- 설정·옵션 조합이 많은 기능
- 조합 전체를 다 테스트하기엔 비용이 큰 경우
- 대부분의 결함은 두 인자의 상호작용에서 발생한다는 경험칙에 근거

테스트 케이스	입력			기대 출력
	도우	토핑	주문	
1	곡물	기본	전화	13000
2	곡물	프리미엄	온라인	12000
3	나폴리	기본	전화	15000
4	나폴리	프리미엄	온라인	14000
5	썬 크리스트	기본	온라인	18000
6	썬 크리스트	프리미엄	전화	19000

경계값 분석 (Boundary Value Analysis)

에러는 경계값 근처에서 빈번히 발생한다. 각 클래스의 하한·상한에서 데이터를 도출해 경계 주위를 꼼꼼히 조사한다.

1-value 경계값

min, max

최솟값과 최댓값만 확인

2-value 경계값

min⁻, min, max, max⁺

경계값과 경계를 벗어난 가장 가까운 값까지

Example 3 : 소수 판정

0~1000 값을 받아 들어 소수를 판정하는 프로그램의 테스트 데이터를 작성 시 예시.

번호	입력 값	기대 결과
1	-1	음수이어서 도움 메시지 출력
2	0	No
4	3	Yes
5	1000	No
6	1001	범위 넘어선 값이어서 도움 메시지 출력
7	2 7	두 개 이상 입력이어서 도움 메시지 출력
8	"r"	문자이어서 도움 메시지 출력
9	공백	공백이어서 도움 메시지 출력

Example — 소수 판정 (0~1000)

- 1-value: 0, 1000 (하한·상한)
- 2-value: -1, 0, 1 / 999, 1000, 1001 (경계와 그 바로 밖)
- 성적 판정·택배 무게처럼 등급 경계가 있는 모든 기능에 동일하게 적용된다

시나리오 테스트 — ATM 현금 인출

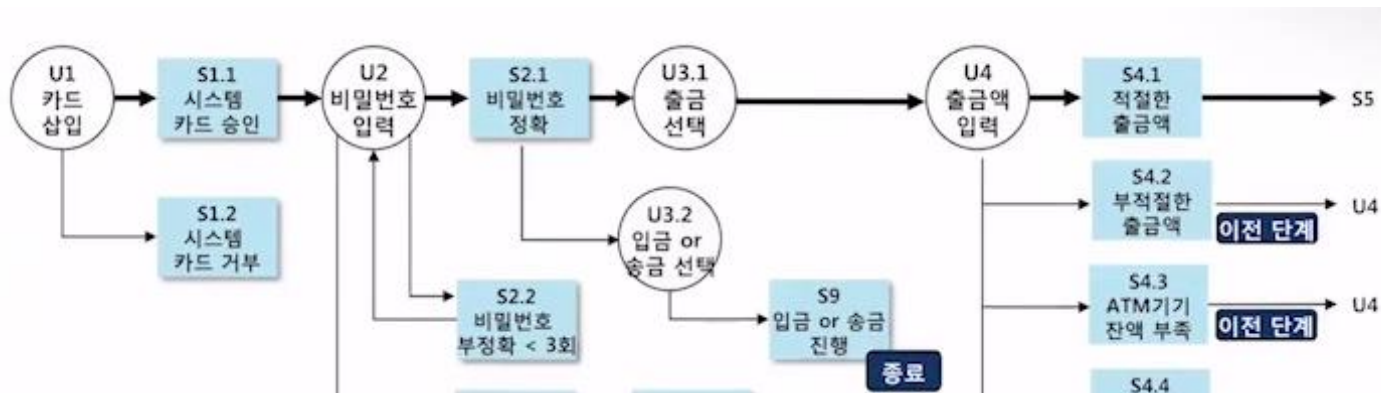
실제 사용 흐름(End-to-End)에 기반해 결함을 발견한다. 시스템·인수 테스트 단계에서 주로 쓴다.

기본(정상) 시나리오

- 계좌에서 성공적으로 현금 인출
- 예) 잔액 \$100, 출금 \$50 → 계좌 \$50, ATM 잔액 차감
- 가장 일반적이고 정상적인 흐름을 나타낸다

대체(예외) 시나리오

- 유효하지 않은 카드
- 비밀번호 오류 2회 / 3회(거래 정지)
- 출금 대신 입금·계좌이체 선택
- 잘못된 출금액(만원 단위 위반)
- ATM 잔액 부족 / 지급 권종 부족
- 1일 한도 초과 / 계좌 잔액 부족



03

화이트박스 테스트 — 구조 기반 커버리지

2부 · 커버리지 종류 · 포함 관계 · MC/DC. 코드의 내부 제어 흐름을 그래프로 파악하고, 문장·분기·조건을 얼마나 검증했는지 측정한다.

테스트 커버리지 종류와 포함 관계

테스트 커버리지는 코드·기능·요구사항이 테스트로 얼마나 검증되었는지 측정하는 지표다.

문장 커버리지	모든 실행 문장(statement)을 한 번 이상	가장 약함
결정(분기) 커버리지	모든 분기(branch)의 참·거짓	if/else 양쪽
조건 커버리지	각 조건(condition)의 참·거짓	복합 조건 각각
결정/조건 커버리지	분기 + 조건을 함께 만족	둘의 결합
MC/DC	각 조건이 결과에 독립적으로 영향	실무 최고 수준
다중 조건 커버리지	모든 조건 조합(2^n)	가장 강함·비쌈

포함 관계: 다중 조건 $\supset$ MC/DC $\supset$ 결정/조건 $\supset$ 결정 $\supset$ 문장

MC/DC — 변형된 조건/결정 커버리지

Modified Condition/Decision Coverage — 각 조건이 다른 조건과 무관하게 결과를 바꿀 수 있음을 보인다. 조건 수 + 1개의 테스트로 검증한다(선형 $n+1$).

복잡도 비교

다중 조건: 지수 2^n

MC/DC: 선형 $n + 1$

→ 조건이 많아질수록 MC/DC가 압도적으로 경제적

표준·산업 활용

- 자동차: ISO 26262 ASIL D 등급 권고
- 항공: DO-178C 표준 Level A 요구사항

Example — AND 조건 2개 (X and Y)

- $N + 1 = 3$ 개 케이스로 각 조건의 독립적 영향을 보인다
- 한 번은 Y를 T로 고정하고 X를 T/F로, 한 번은 X를 T로 고정하고 Y를 T/F로
- OR 조건도 대칭적으로 3개 케이스(각 조건을 F로 고정)로 검증한다

IF ((X > 1) AND (Y == 1))			
TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F
TC4	F	F	F

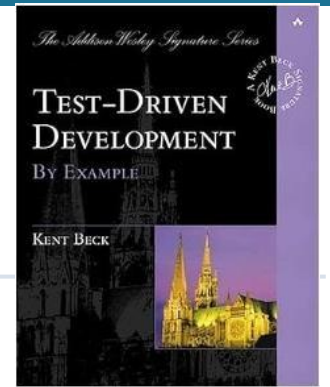
TC	(X>1)	(Y==1)	(X>1) AND (Y==1)
TC1	T	T	T
TC2	T	F	F
TC3	F	T	F

04

TDD 사이클 — RED · GREEN · REFACTOR (Python)

실패하는 테스트를 먼저 보고, 최소 구현으로 통과시키고, 동작을 바꾸지 않고 구조만 정리한다.

TDD 한 사이클 — RED · GREEN · REFACTOR (Python)



RED

실패하는 테스트를 먼저

만지는 것 · tests/ 만 — 구현(src/)은 건드리지 않는다

끝나는 신호 · 테스트가 “의도대로” 실패

GREEN

최소 구현으로 통과

만지는 것 · src/ 만 — 요구사항에 없는 기능은 금지

끝나는 신호 · 테스트가 전부 통과

REFACTOR

동작 그대로 구조 정리

만지는 것 · 중복 제거 · 이름 정리 — 계약 불변

끝나는 신호 · 여전히 GREEN 유지

```
# RED — tests/test_calc.py
def test_add(): assert add(2, 3) == 5
```

```
# GREEN — src/calc.py
def add(a, b): return a + b
```

pytest 핵심 — 이 과정에서 쓰는 것만

기능	pytest 사용법
테스트 정의	<code>def test_name(): ...</code>
값 동등	<code>assert expected == actual</code>
조건 참	<code>assert condition</code>
예외 발생	<code>with pytest.raises(ValueError): ...</code>
사전/사후 실행	<code>@pytest.fixture (setup/teardown)</code>
매개변수 반복	<code>@pytest.mark.parametrize("a,b", [...])</code>
커버리지	<code>pytest --cov=src --cov-report=term-missing</code>

실행 명령

`pytest -q`

전체 실행

`pytest tests/test_cart.py -q`

파일 단위 실행

`pytest -k test_vip`

이름으로 선택 실행

`pytest --cov=src`

커버리지 측정

Test 프롬프트 엔지니어링 — 5요소

막연한 “고쳐줘”는 범위가 폭주한다. 아래 5요소를 명시할수록 결과가 안정적이다.

1	역할/관점	어떤 시각으로 볼지	“보안 리뷰어 관점에서”
2	맥락	대상·제약·배경	“calc 함수만, 동작 변경 없이”
3	작업	구체적 단일 동작	“스멜을 심각도 순으로 나열”
4	제약	하지 말 것·범위	“파일 수정 말고 보고만”
5	출력 형식	받을 모양	“표: 위치 문제 위험 제안”

Dual-Track — 두 트랙으로 나눠 돌린다

한 화면짜리 앱이라도 “계산 규칙(Logic)”과 “화면 동작(UI)”은 성격이 다르다.

구분	Logic 트랙	UI 트랙
검증 대상	순수 함수(입력→출력) 계산 규칙	화면 렌더링·사용자 입력 흐름
이번 실습	subtotal · 할인 · VIP 계산	주문 폼 · 최종 금액 표시
속도/안정성	매우 빠르고 결정적	상대적으로 느리고 환경 영향
도구	pytest 단위 테스트	Flask test_client · Simple Browser · 브라우저 MCP

혼자 도는 5단계 RED → GREEN 루프



반복의 원칙 — 한 번에 요구사항 하나. RED→GREEN→커밋을 마치기 전에 다음으로 넘어가지 않는다. 실패 로그는 매번 새로 확인한다.

05

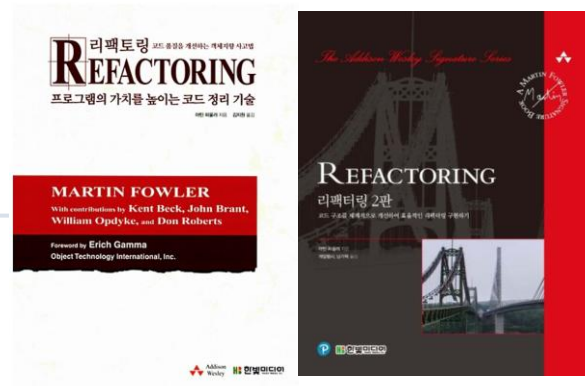
Code Smell & Refactoring

24가지 코드 냄새 · 리팩토링 기법 · 클린코드 원칙 · SOLID · 정적 분석 (Python). 외부 동작을 바꾸지 않으면서 내부 구조를 개선한다.

24가지 코드 냄새 (Fowler)

1 이해하기 힘든 이름	2 중복 코드	3 긴 함수	4 긴 매개변수 목록
5 전역 데이터	6 가변 데이터	7 뒤엎힌 변경	8 산탄총 수술
9 기능 편애	10 데이터 뭉치	11 기본형 집착	12 반복되는 Switch
13 반복문	14 정의 없는 요소	15 추측성 일반화	16 임시 필드
17 메시지 체인	18 중재자	19 내부자 거래	20 거대한 클래스
21 대안 클래스들	22 데이터 클래스	23 상속 포기	24 주석

리팩토링 개요와 절차 (Python)



“리팩토링은 외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 방법이다. 버그가 끼어들 가능성을 최소화하면서 코드를 정리하는 정형화된 방법이다.” *"Inspect and Adapt"* (점검하고 적응하라)

1. 테스트 코드 작성

2. Code Smell 감지

3. 리팩토링 기법 선택

4. 리팩토링 수행

5. 테스트 재실행(녹색 확인)

VS Code Python 리팩토링 도구

- Pylance (언어 서버): 이름 바꾸기·자동 완성·타입 힌트
- Ctrl+. : Quick Fix / Refactor (메서드 추출 등)
- mypy: 정적 타입 검사
- F2: 심볼 이름 바꾸기 (Rename Symbol)
- ruff / black: 스타일 자동 정렬
- pytest --cov: 커버리지로 안전망 확인

3의 법칙 (Rule of Three) — Python 예제

돈 로버츠(Don Roberts): ① 처음엔 그냥 한다 ② 두 번째 중복은 일단 진행한다 ③ 세 번째 반복되면 리팩토링한다.

✗ 세 번째 중복 (냄새)

```
class UserService:
    def get_user_full_name(self, first, last):
        return f'{first} {last}'

    def get_admin_full_name(self, first, last):
        return f'{first} {last}' # 중복

    def get_guest_full_name(self, first, last):
        return f'{first} {last}' # 세 번째!
```

✓ 공통 로직 추출

```
def format_full_name(first, last):
    return f'{first} {last}'

class UserService:
    def get_user_full_name(self, f, l):
        return format_full_name(f, l)
    def get_admin_full_name(self, f, l):
        return format_full_name(f, l)
    # 이름 관련 로직을 한 곳으로 위임
```

쓰레기 줍기 리팩토링 (Python) — 매직 넘버 제거

코드베이스에 산재한 작고 명확한 문제(디버그 출력·불필요 변수·미사용 코드·매직 넘버)를 점진적으로 정리한다.

Before

```
class TaxCalculator:
    def calculate_tax(self, price):
        # 0.08이 무엇인지 불명확
        return price * 0.08
```

After

```
class TaxCalculator:
    TAX_RATE = 0.08 # 의미 있는 상수

    def calculate_tax(self, price):
        return price * self.TAX_RATE
```

변경점: 매직 넘버 0.08을 TAX_RATE 상수로 추출해 의미를 명확히 하고 유지보수성을 높였다. 디버그 print·미사용 함수·불필요 변수 제거도 같은 원리다.

변화를 위한 리팩토링 — 조건문을 디스패치로 (Python)

새 유형이 추가될 때마다 if/elif를 늘리는 구조는 변경에 취약하다. 덕서너리 디스패치로 확장에 열고 변경에 닫는다(OCP).

✗ 변경에 취약

```
class OrderProcessor:
    def process(self, order_type):
        if order_type == 'online':
            print('Processing online order')
        elif order_type == 'in_store':
            print('Processing in-store order')
        # subscription 추가 시 또 elif...
```

✓ 덕서너리 디스패치

```
class OrderProcessor:
    def __init__(self):
        self._handlers = {
            'online': self._online,
            'in_store': self._in_store,
        }
    def process(self, order_type):
        handler = self._handlers.get(order_type)
        if handler: handler()
```

Boy Scout Rule & 클린코드 핵심 원칙 (Python)

 “캠프장은 처음 왔을 때보다 더 깨끗하게 해놓고 떠나라.” — 손댄 코드는 조금이라도 더 깨끗하게 남긴다.

표준 컨벤션 준수

코딩 표준·설계 가이드를 지킨다 (Python: PEP 8, ruff / flake8)

KISS

단순한 것이 효율적이다. 복잡한 로직은 함수로 분리한다

근본 원인 분석

표면만 고치면 반복된다. 정적 분석으로 패턴을 파악한다

DRY

중복은 유지보수 지옥이다. 공통 유틸로 분리한다 (ruff·pylint 중복 탐지)

객체 지향 설계 5원칙 — SOLID

S

단일 책임 (SRP)

하나의 클래스는 하나의 책임만 가진다

O

개방-폐쇄 (OCP)

확장에는 열려 있고 변경에는 닫혀 있다

L

리스코프 치환 (LSP)

파생 클래스는 기반 클래스를 대체할 수 있어야 한다

I

인터페이스 분리 (ISP)

쓰지 않는 메서드에 의존하지 않는다

D

의존 역전 (DIP)

추상화에 의존하고 구체적인 것에 의존하지 않는다

SRP 예시 — 책임을 분리한다 (Python)

클래스는 단 하나의 책임만 가진다 — 변경의 이유가 하나여야 한다. 직렬화·표현 책임은 별도로 분리한다.

✗ SRP 위반

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...
    def to_json(self): ... # 직렬화?
    def to_xml(self): ... # 직렬화?
    def to_html(self): ... # 표현/UI?
```

✓ SRP 준수

```
class Resume:
    def introduce_me(self): ...
    def get_grade(self): ...

class Converter(Protocol):
    def convert(self, r: Resume) -> str: ...

class JsonConverter: # Converter 구현
    def convert(self, r): ...
```

OCP · LSP · ISP · DIP — Python 적용 요약

원칙	위반 신호	Python 해법
OCP	새 유형마다 if/elif 추가	추상 기반 클래스(ABC)·Protocol + 딕셔너리 디스패치
LSP	자식이 부모 계약을 깬 (fly 못하는 Ostrich)	공통은 Bird, 능력은 Flyable 인터페이스로 분리
ISP	쓰지 않는 메서드 강제 구현 (Penguin.fly)	Bird / FlyableBird로 인터페이스 분리
DIP	구체 클래스에 직접 의존 (EmailService)	MessageService 추상화 + 생성자 주입(DI)

Python에서는 `abc.ABC`·`typing.Protocol`로 추상화를 표현하고, 생성자 주입으로 의존을 역전한다. 다형성과 덕 타이핑이 SOLID 적용을 자연스럽게 해 준다.

좋은 네이밍의 원칙 (Python)

의도를 표현하라

❌ `d` # 경과 일수

✅ `elapsed_days`

그릇된 정보 피하기

❌ `account_list` # List 아닐 수도

✅ `accounts`

의미 있는 구분

❌ `a1, a2 / product_info`

✅ `source, destination`

검색 가능한 이름

❌ `if s == 5:`

✅ `WORK_DAYS = 5`

클래스 = 명사

❌ `class Manager / Processor`

✅ `class Customer / Account`

함수 = 동사

❌ `name() / age()`

✅ `get_name() / is_valid()`

Python 네이밍 & 스타일 — PEP 8

구분	규칙 (PEP 8)	예시
클래스	PascalCase	CalculatorService
함수 / 메서드	snake_case	calculate_total()
변수	snake_case	elapsed_time
상수	UPPER_SNAKE_CASE	MAX_SIZE
비공개 멤버	선행 밑줄	_internal_state
모듈 / 패키지	짧은 소문자	cart, utils.math_utils
들여쓰기	스페이스 4칸	탭 대신 스페이스
최대 라인 길이	79 (docstring 72)	black / ruff 로 자동 정렬

 **VS Code:** ruff / black / flake8 로 스타일을 자동 강제할 수 있다.

함수 설계 — 작게, 하나만, 한 수준으로

1 작게 만들어라

20줄 이내가 이상적. 들여쓰기 1~2단계 이내

2 한 가지만 해라

함수 이름으로 설명되는 한 가지 작업만

3 추상화 수준 통일

고수준(비즈니스)과 저수준(I/O)을 섞지 말 것

4 서술적 이름

길고 서술적인 이름이 짧고 의미 없는 이름보다 낫다

5 인수를 최소화

이상 0개, 최대 3개. 초과 시 매개변수 객체 고려

6 부수 효과 없애기

숨겨진 변경 금지. 명령과 조회를 분리(CQS)

데미터 법칙 — 최소 지식 원칙 (Python)

Law of Demeter — 낯선 이에게 말 걸지 않는다. 연쇄 호출(.으로 이어지는 기차)은 결합도의 냄새다.

✖ 나쁜 예 (위반)

```
city = order.customer \
      .address \
      .city
```

- order→customer→address→city 내부 구조를 다 안다
- 구조가 바뀌면 호출부가 전부 깨진다

✔ 좋은 예 (준수)

```
city = order.customer_city()
# Order 내부에서 위임 처리:
# return self._customer.address.city
```

- 외부는 Order 하나만 알면 된다
- 내부 구조 변경이 호출부에 새어 나가지 않는다

위임 메서드로 감싸 내부 구조를 숨긴다 — 객체는 자신의 직접 이웃하고만 대화한다.

Clean Error Handling (Python)

원칙	나쁜 코드 ❌	좋은 코드 ✅
오류 코드 대신 예외	if delete_page(p) == E_OK:	try: delete_page(p) / except:
None 반환하지 않기	return None # 책임 떠넘김	빈 리스트 · 명시적 예외 반환
None 전달하지 않기	do_something(None)	가드 절 · Optional 명시
예외에 의미 담기	raise Exception("error")	raise StorageError(...) from e

```
# 예외 체이닝으로 원인 보존 (raise ... from e)
def read_config(path):
    try:
        return _parse(path)
    except OSError as e:
        raise StorageError(f'설정 읽기 실패: {path}') from e
```

코드스멜 주요 유형과 처방

중복 코드

같은 코드 두 곳 이상 → 함수 추출 / 공통 유틸

긴 함수

20줄 초과 → 기능별 분리

거대한 클래스

너무 많은 책임 → SRP 적용, 분리

긴 매개변수 목록

3개 초과 → 매개변수 객체 도입

죽은 코드

미사용 코드 → 즉시 삭제 (Git이 기억)

매직 넘버

의미 불명 값 → 이름 있는 상수로 추출

깊은 중첩

3단계 이상 → Early Return / 함수 추출

부적절한 이름

a, tmp, data → 의도를 드러내는 이름

정적 분석으로 코드스멜을 자동 탐지 (Python)

정적 분석은 실행 없이 소스코드를 분석해 버그·스타일 위반·복잡도 문제를 찾아낸다.

도구	탐지 내용	연동 방법
Pylance	실시간 타입·문법 경고	VS Code Python 확장 내장
ruff	스타일·버그·복잡도 (초고속)	ruff check . / pre-commit
flake8	코딩 규칙 (PEP 8·복잡도)	flake8 + 설정 파일
mypy	정적 타입 검사	mypy src
bandit	보안 취약 패턴	bandit -r src
pytest-cov	테스트 커버리지 측정	pytest --cov=src

```
ruff check . && mypy src && pytest --cov=src — 한 파이프라인으로 정적 분석을 자동 실행한다.
```

코드스멜 → 도구·규칙 → 클린코드 원칙 (Python)

코드스멜	도구 & 규칙 (Python)	클린코드 원칙
매직 넘버	ruff / pylint: magic-value	의도 표현 · 명명
빈 except 블록	ruff: BLE001 (blind-except)	오류 처리
미초기화·미사용 변수	ruff / pyflakes: F841	안전성 · 죽은 코드
None 역참조 위험	mypy: optional 검사	오류 처리
미사용 코드	ruff: F401 · vulture	죽은 코드 제거
중복 코드	pylint: duplicate-code	DRY 원칙
과도한 복잡도	ruff: C901 (mccabe)	함수 단순화
타입 불일치	mypy: 타입 검사	코드 명확성

QA를 위한 프롬프트 엔지니어링 — 5요소

막연한 “고쳐줘”는 범위가 폭주한다. QA 프롬프트는 아래 5요소를 명시할수록 결과가 안정적이다.

- | | | |
|---|-------|------------------------|
| 1 | 역할/관점 | “보안 리뷰어 관점에서” |
| 2 | 맥락 | “calc 함수만, 동작 변경 없이” |
| 3 | 작업 | “스멜을 심각도 순으로 나열” |
| 4 | 제약 | “파일 수정 말고 보고만” |
| 5 | 출력 형식 | “표: 위치 문제 위험 제안” |

출력 형식을 지정하면 결과를 그대로 QA 리포트·Skill 절차로 재활용할 수 있다 — 형식은 자동화의 입력이 된다.

작업별 프롬프트 패턴 — 진단 · 리뷰 · 리팩터 · 테스트

진단

«대상»의 코드 스멜·중복·복잡도를 찾아줘. 수정 말고 [위치|유형|근거|난이도] 표로, 우선순위 순으로 정리해줘.

리뷰

방금 변경을 «보안|회귀|성능» 관점으로만 리뷰해줘. 동작이 바뀔 지점과 그걸 잡을 테스트가 없는 부분을 우선순위로.

리팩터

«함수»의 외부 동작을 바꾸지 말고 내부 구조만 개선해줘. 먼저 계획을 보여주고 승인 후 적용, 매 단계 후 테스트 실행.

테스트

«함수»의 테스트 갭을 찾아줘(경계·예외·중복). 빠진 케이스를 먼저 보여주고, 합의된 케이스만 테스트로 추가해줘.

범위 폭주를 막는 한 줄 — “먼저 계획을 보여주고 승인 후 진행. 요청하지 않은 변경은 하지 마.”

06

Concept-to-Code (C2C) 딜리버리 프레임워크 와 Traceability

실패 없는 소프트웨어를 위한 해석학적 접근. 개념·불변식에서 코드까지 연결고리를 끊지 않는 개발.

왜 우리는 항상 늦게 실패하는가

“실패 조건이 정의되지 않으면, 성공은 함의될 수 없다.” 기획서엔 기능은 있지만 실패 조건이 없다— 이것이 “늦은 실패”의 본질이다.

원인 1

기획은 있는데 설계는 없다

요구사항 문서 = 기능 목록. 실패 조건이 없어 개발자마다 해석이 다르다

원인 2

테스트는 항상 늦게 추가된다

“구현 먼저, 테스트 나중” → 테스트가 구현의 하인이 되어 비즈니스 의미를 잃는다

원인 3

UI/Logic 분리의 오해

UI가 비즈니스 규칙을 포함 → 어느 것도 독립적으로 테스트할 수 없다

Dual-Track TDD: 실패 조건 함의 → RED 테스트 → 구현. 실패 조건을 먼저 정의하면 “늦은 실패”가 “이른 함의”로 바뀐다.

왜 설계 의도는 코드에서 사라지는가 — Semantic Erosion

“완벽한 요구사항 문서를 작성했다. 6개월 후 코드를 보니 설계 의도는 온데간데없고 스파게티만 남아 있다.” — 시스템은 시간이 지날수록 의미적 일관성을 잃는다.



문서-코드 단절

개념 분석 산출물이 코드와 직접 연결되지 않는다



테스트 수준 혼재

기술 단위 테스트만 작성 — 비즈니스 의미 검증이 없다



UI의 도메인 잠식

UI 요구사항이 도메인 모델을 오염시킨다



아키텍처 우선 개발

아키텍처가 개념보다 먼저 결정된다

문제 인식과 의사결정 파이프라인

문제 인식이란 “언제 시스템이 실패했다고 말해야 하는가”를 정의하는 것 — “무엇을 만들까”가 아니라 실패 조건부터 정한다.

문제인식

의미화

온톨로지

디지털화

논리화

정규화

검증화

추상화

최적화

실행·결정

❌ 해결책 중심 (나쁜 문제 인식)

- 책을 빌리는 기능이 필요하다
- 로그인 화면을 만들자
- 검색 기능을 추가하자

✅ 실패 조건 중심 (올바른 문제 인식)

- 연체·한도 초과 시 → 대출 거부
- 이메일 오류·비밀번호 오류 각각 처리
- 검색어 0자·특수문자·결과 없음 정의

온톨로지와 ECB 분석 모델

온톨로지는 “세계를 어떤 개념·관계로 나누어 이해할 것인가”를 정의하는 명시적 모델이다. ECB는 객체를 역할로 나누는 분석 도구다.



Entity (엔티티)

시스템이 장기적으로 관리하는 도메인 정보

예: Member · Book · Loan · Order

💡 판단하지 않는다. 상태를 유지한다.



Control (제어)

유스케이스를 수행하는 흐름 제어·업무 조정

예: LoanPolicy · OrderService

💡 판단한다. 조율한다. 상태는 최소화.



Boundary (경계)

시스템과 외부 액터 사이의 접점

예: LoanUI · LoginAPI · Payment

💡 비즈니스 규칙을 포함하지 않는다.

⚠️ ECB는 MVC·Spring Controller와 다르다. 분석 개념 도구이지 코딩 규칙이 아니다 (Ivar Jacobson, OOSE 1992).

Concept-to-Code Traceability — 4계층 모델

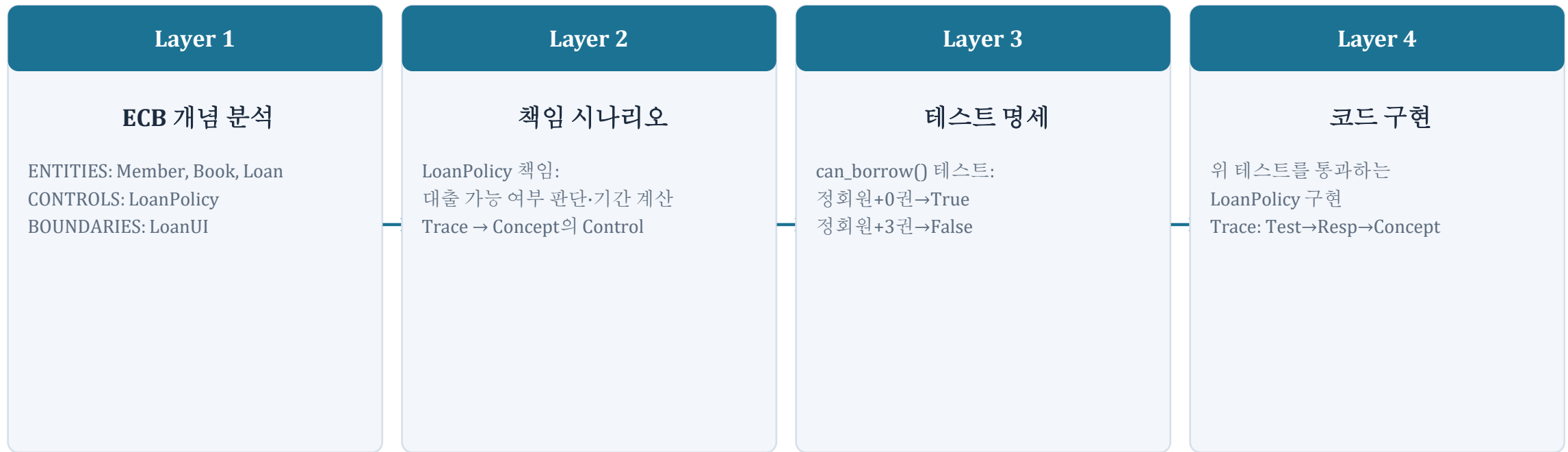
문제 영역의 개념(Concept)과 코드(Code) 사이의 의미적·구조적·행위적 연결을 명시적으로 유지·검증·진화 가능한 상태 — 의미 보존이 목표다.



핵심 성질: 완전성(Completeness) · 일관성(Consistency) · 검증 가능성(Verifiability) · 진화 가능성(Evolvability)

프롬프팅 체인 — Traceability의 현대적 구현

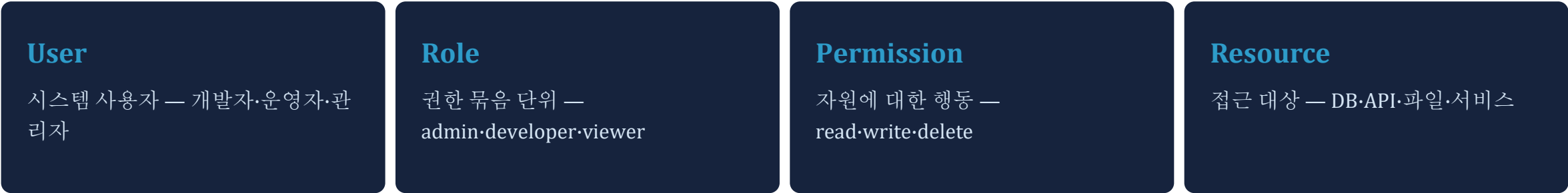
프롬프트 엔지니어링 = C2C Traceability를 구현하는 현대적 방법론. 각 프롬프트가 상위 계층을 명시적으로 참조하면 Chain-of-Prompts = Traceability Chain 이 된다.



각 프롬프트가 상위 계층을 명시적으로 참조 → Traceability가 자동으로 확보된다.

역할 기반 접근 제어(RBAC) — ECB와의 연결

ECB에서 Control이 규칙을 실행하고 Boundary가 외부와 통신한다면, RBAC는 “누가 어떤 Control을 실행할 수 있는가”를 결정하는 접근 제어 레이어다.



E · Entity	User·Role·Permission·Resource → RBAC 핵심 데이터 보관
B · Boundary	로그인 폼·API 엔드포인트 → Role 정보를 받아 Control로 전달
C · Control	AuthService·PermissionChecker → Role-Permission 매핑·허용/거부

RBAC 설계 — 권한에 따른 메뉴 가시성 (Python)

역할	대시보드	사용자관리	설정	삭제
admin	✓	✓	✓	✓
developer	✓	✓	✗	✗
viewer	✓	✗	✗	✗
guest	⚠	✗	✗	✗

UX Contract — “보인다 / 안 보인다”

- admin: 사용자관리 보임·삭제 활성화
- developer: 사용자관리 보임·삭제 비활성
- viewer: 대시보드만 보임·관리 메뉴 숨김

```
from enum import Enum
```

```
class Role(Enum):
    ADMIN = 'admin'; DEVELOPER = 'developer'
    VIEWER = 'viewer'

MENU_VISIBILITY = {
    Role.ADMIN: {'dashboard': True, 'user_mgmt': True,
                 'settings': True, 'delete_btn': True},
    Role.DEVELOPER: {'dashboard': True, 'user_mgmt': True,
                     'settings': False, 'delete_btn': False},
    Role.VIEWER: {'dashboard': True, 'user_mgmt': False,
                  'settings': False, 'delete_btn': False},
}
```

```
def is_visible(role, menu):
    return MENU_VISIBILITY.get(role, {}).get(menu, False)
```

RBAC + TBAC 하이브리드 모델



RBAC — 역할 기반

역할(Superadmin·Admin·Manager·User·Viewer)에 따라 기능 접근을 제어한다. 예: Admin만 사용자 관리, Viewer는 읽기만. 권한 기반 작업 시 유지보수가 수월하다.



TBAC — 팀 기반

사용자가 소속된 팀에 따라 데이터 접근 범위를 제어한다. 예: R&D 팀 소속은 R&D 게시판·문서만 접근.



ABAC — 속성 기반

팀 소속이라는 “속성”을 기준으로 접근을 제어한다. TBAC를 일반화한 형태.

RBAC × TDD × Gemini CLI — 실전 워크플로우

RBAC를 TDD로 구현할 때 Gemini CLI를 활용하는 전체 흐름 — 역할 개념 → 실패 테스트 → 최소 구현 → 리팩토링까지 의미 소실 없이 추적한다.

① 역할 정의

ECB Entity로
Role·Permission 데이터 구조
설계

② 실패 테스트

권한 거부/허용 실패 테스트
먼저 → pytest FAIL 확인

③ 최소 구현

메뉴 가시성 딕셔너리 +
is_visible() 함수

④ 리팩토링

JWT claim에서 Role 읽는 구
조로 개선·역할 계층화

⑤ Git 커밋

Role 개념→테스트→코드 연
결(C2C) 확인

RED — 실패 테스트 예시

```
def test_viewer_cannot_see_user_mgmt():
    assert not is_visible(Role.VIEWER, 'user_mgmt')
def test_admin_can_see_all_menus():
    assert all(is_visible(Role.ADMIN, m) for m in MENU_VISIBILITY[Role.ADMIN])
```

C2C의 정의와 4계층 검증 구조

C2C(Concept-to-Code) — 개념·불변식에서 코드까지 연결고리를 끊지 않는 것. 임의의 구현 줄에서 위로 올라가면 반드시 불변식에, 임의의 불변식에서 아래로 내려가면 테스트·구현 줄에 닿는다. 이 양방향 도달 가능성이 곧 C2C다.

계층	이름	질문	ECB 대응
Layer 1	Invariant (불변식)	무엇이 절대 바뀌면 안 되는가	수학적 진실
Layer 2	UI Contract Test	UX가 올바르게 반응하는가	Boundary
Layer 3	Domain Test	비즈니스 규칙이 작동하는가	Control
Layer 4	Implementation Unit	단위 알고리즘이 정확한가	Entity

Layer 1(불변식)은 사슬의 정점 — 그 불변식은 어디서 오는가? 이 질문이 제품 계약(3장)으로 이어진다.

계약 ID(추적의 못)와 추적성 매트릭스

RED

테스트 이름·주석에 계약 ID

GREEN

구현 줄마다 계약 ID 주석

COMMIT

커밋 메시지에도 계약 ID

금지

ID 없는 동작은 구현하지 않는다

계약ID	의미	검증 테스트	구현 위치
INV-1	subtotal = $\sum \text{price} \times \text{qty}$	test_subtotal_sums_price_times_qty	subtotal() return
INV-2	경계 포함 문턱 할인	test_threshold_applies_at_50000	apply_threshold_discount()
INV-3	문턱→VIP 순서·금액	test_vip_order_and_value	final_total()
INV-4	$0 \leq \text{final} \leq \text{subtotal}$	test_discount_never_increases	final_total() return
E-1 / E-2	None→TypeError / 음수→ValueError	boundary::test_none / _negative	subtotal() 가드·루프

구현에는 있으나 매트릭스에 없는 코드 줄 = “정체불명 코드” → 제거하거나 계약화한다. 과잉 구현을 원천 차단한다.

AC · Invariant · Error Contract — 구분과 승격

구분	의미	예
AC (수용 기준)	사용자가 받아들일 조건 — Given-When-Then	“5만원 이상이면 할인된다”
Invariant (INV)	어떤 입력에서도 깨지면 안 되는 규칙	“할인 후 금액이 원금보다 커질 수 없다”
Error Contract (E)	잘못된 입력에 대한 명시적 오류 규칙	“qty가 음수이면 ValueError”

증거 수준 — 희망과 증거를 섞지 않기

- L0 — 협상 불가 외부 제약 / 과거 행동+수치 동반
- L1 — 고객 발화·실측으로 뒷받침되는 항목
- L2 — 근거 약한 추정 → 우선순위 하락·추가 검증

AC → INV 승격의 예

AC-1 “5만원 이상 10% 할인”에서 INV-2가 나오지만, Mom Test로 “절대 발생하면 안 되는 결과는?”을 물어야 INV-4: $0 \leq \text{final} \leq \text{subtotal}$ 이라는 더 높은 불변식이 나온다.

AC에서 계약을 뽑아내는 4가지 규칙

①

AC 그 자체 → 테스트 조건

Given-When-Then을 그대로 테스트로 번역. 예) 50000 적용 / 49999 미적용(경계 2행)

②

숨은 항상성 → INV-*

어떤 입력에서도 깨지면 안 되는 규칙. 예) INV-2: $\text{amount} \geq 50000 \rightarrow \text{round}(\times 0.9)$

③

잘못된 입력 → E-*

명시적 오류 규칙 — 조용한 실패 금지. 예) 음수 qty → ValueError

④

AC에 없는 기능 → OOS-*

만들지 않는다 — 과잉 구현 차단. 예) 쿠폰·배송비·포인트

Invariant는 AC보다 강한 도메인 계약이다. 그 간극을 메우는 것이 Mom Test — “할인했는데 원래보다 비싸지면 안 된다” → INV-4.

Dual-Track TDD와 ARRR 실행 리듬

단계	의미	TDD	완료 신호
A — Ask	테스트로 시스템에 질문한다	RED	의도된 FAIL
R — Respond	통과하는 최소 코드로 답한다	GREEN	전부 PASS
R — Refine	중복 제거·이름 정리로 다듬는다	REFACTOR	계약 불변·GREEN
R — Repeat	다음 계약 ID로 다음 질문	다음 RED	회귀(pytest -q)

이중성 ① 프로세스 수준 — Discovery(무엇이 옳은가) || Delivery(옳게 만든다).

이중성 ② 테스트 아키텍처 — Track A(UI/Boundary) || Track B(Domain/Entity).

△ 용어 주의 — 네 글자 ARRR(Ask·Respond·Refine·Repeat)은 실행 리듬, 다섯 글자 AARRR(Acquisition...Revenue)은 그로스 퍼널 지표. 별개의 개념이다.

ARRR 예제 — 숫자 더하기 계산기 (Python)

빈 문자열 → 0, 단일 숫자 → 그 값, 쉼표 구분 합계. 이후 세미콜론 구분자를 Repeat로 추가한다.

A · Ask (RED)

```
def test_empty_returns_zero():
    assert add('') == 0
def test_single_number():
    assert add('5') == 5
def test_two_numbers():
    assert add('3,5') == 8
```

R · Respond (GREEN)

```
def add(numbers):
    if not numbers:
        return 0
    return sum(
        int(n) for n in numbers.split(',')
```

R · Refine (REFACTOR)

```
# 구분자 분리 로직을 함수로 추출(SRP)
def _split(numbers): ...
```

R · Repeat (새 RED)

```
def test_semicolon_delimiter():
    assert add('3;7') == 10 # 새 요구사항
```

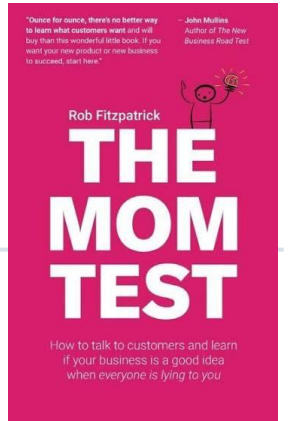
Dual-Track TDD 5단계 사이클 — 두 트랙의 맞물림

단계	Discovery 트랙	Delivery 트랙	ARRR	신호
① 가설 수립	계약·골든셋·목표 수치 정의, ID 부여	(대기)	—	계약 표
② Red	가설을 검증 가능한 조건으로	ID마다 실패 테스트(tests/만)	Ask	의도된 FAIL
③ Green	구현이 가설을 만족하는지 확인	통과 최소 구현, 구현 줄에 ID 주석	Respond	전부 PASS
④ Refactor	평가 기준이 정밀해질 지점 식별	중복 제거·SSOT 추출, 계약 불변	Refine	여전히 GREEN
⑤ 합류/반복	골든셋·지표로 전체 검증	E2E 스모크 + 다음 ID로 회귀	Repeat	매트릭스 완성

닫힌 루프 — Discovery의 산출물이 Delivery의 테스트 픽스처가 되고, Delivery의 계측이 Discovery의 다음 가설이 된다.

The Mom Test 란?

The Mom Test(엄마 테스트)는 롭 피츠패트릭(Rob Fitzpatrick)이 저술한 동명의 책에서 유래한 개념으로, 사업 아이디어가 유효한지 검증하기 위해 고객과 진솔한 대화를 나누는 방법론



스타트업·제품 아이디어 검증에서 쓰이는 인터뷰 기법. 상대가 ‘좋은 사람이라서’ 거짓 긍정(칭찬·공감)을 해도 속지 않도록 질문하는 방법이다.

Mom Test = “사람의 말이 아니라 행동을 믿는 인터뷰 방법”

계약(INV·E·AC)의 원천은 “고객이 실제로 어떻게 행동했는가”다. Mom Test는 그 근거를 캐내는 도구다.

잘못된 질문 vs 올바른 질문

✖ 잘못된 질문

“제가 만든 설비 모니터링 앱 어때요?”

→ 칭찬만 들을 것

“이런 기능이 있으면 사용하시겠어요?”

→ 가짜 약속만 받을 것

“이 앱에 얼마 지불하시겠어요?”

→ 현실성 없는 답변만 받을 것

✔ 올바른 질문

“최근 설비 점검하며 가장 답답했던 순간은?”

→ 진짜 문제 발견

“지금은 그 상황을 어떻게 해결하고 계세요?”

→ 현재 우회 방법 파악

“그 방법의 어떤 점이 불편하세요?”

→ 개선 포인트 확인

핵심 원칙 3가지와 피해야 할 3가지

핵심 원칙 3가지

- 당신의 아이디어에 대해 말하지 마라
- 미래나 가정에 대해 묻지 마라
- 과거의 구체적인 행동에 대해 물어라

피해야 할 3가지 데이터

칭찬 (Compliments)

“정말 좋은 아이디어네요!” → 행동으로 이어지지 않음

가상의 약속 (Hypotheticals)

“있으면 꼭 쓸 것 같아요!” → 실제로는 안 씀

일반론 (Generics)

“보통 사람들은 좋아할 거예요” → 추측일 뿐

올바른 질문하기 — 과거·투자·해결 시도

과거의 구체적 사건

“지난주 설비 점검 때 어떤 일이 있었나요?”
“정확히 어떤 절차를 거치셨나요?”
“얼마나 시간이 걸렸나요?”

돈·시간 투자 확인

“현재 도구에 비용을 지불하시나요?”
“배우는 데 시간을 얼마나 쓰셨나요?”
“팀원들도 다 쓰고 있나요?”

해결 시도 이력

“이 문제를 해결하려 시도해 보셨나요?”
“어떤 도구들을 써보셨나요?”
“왜 계속 쓰지 않으셨나요?”

인터뷰 진행하기 — Mom Test 실전 (Gemini 역할극)

실무에서는 인터뷰 메모를 스스로 만들어야 한다. Gemini에게 질문지를 짜게 하고, 고객 역할을 맡겨 연습한다.

① 인터뷰 가이드 생성

당신은 Mom Test 규칙을 엄격히 적용하는
인터뷰 설계자다.

job: "주문 담당자가 할인 금액을 오류 없이 확정"

1. 과거 행동을 캐는 질문 8개(미래·의견 금지)
2. 각 질문 옆에 '캐내려는 계약 후보' 한 줄
3. 반드시 포함:
'절대 발생하면 안 되는 결과는?' → INV 후보
'잘못 입력하면 뭐가 일어났나?' → E 후보
4. 물으면 안 되는 유도 질문 3개를 반례로

② 역할극 — Gemini가 고객이 된다

지금부터 당신은 온라인 쇼핑몰 주문 담당자다.

- 과거에 실제 겪은 일만 말한다.
해결책을 제안하지 않는다.
- 내가 유도 질문('~하면 되죠?')을 하면,
먼저 '유도 질문입니다'라고 지적하고
어떻게 바꿀지 알려준다.
- '인터뷰 종료' → [인터뷰 메모] 형식 정리.
없는 사실을 지어내지 않는다.
첫 질문을 기다린다.

인터뷰 메모를 계약으로 변환

[인터뷰 메모] — 실습 입력

- A 3개(12,000)+B 1개(30,000)=66,000. 5만원 넘으면 10% 할인. 기대 59,400 (엑셀 계산 10분)
- VIP는 문턱 할인 후 추가 5%. 순서 문턱→VIP 고정 (순서 뒤바뀔 클레임 1건)
- 수량 -1 입력 시 합계 0원 표시(기대: 오류) / items 없이 제출 시 500 에러
- 쿠폰·배송비: 언급 없음 / “50,000원 정확히” 포함 여부 미확인

Discovery 트랙 — 6단계

- Step1. 각 항목을 증거 수준(L0/L1/L2)으로 분류
- Step2. Invariant/Error/AC/Out-of-Scope 추출
- Step3. ID 부여 (INV-n·E-n·AC-n·OOS-n)
- Step4. 테스트 가능한 문장으로 변환
- Step5. AC는 Given-When-Then으로
- Step6. 출력: ID|계약|증거수준|계층|테스트 아이디어

규칙: 근거 없는 동작을 계약ID로 만들지 마라.

확정 계약표를 docs/PRD.md에 고정 (SSOT)

ID	계약 (불변식 / 에러)	증거 수준	계층
INV-1	<code>subtotal(items) == Σ(price × qty)</code>	— (수학적 정의)	Entity
INV-2	<code>amount ≥ 50000 → round(×0.9) / 미만 그대로</code> (경계 포함)	L1	Entity
INV-3	문턱 할인 후 VIP면 <code>round(×0.95)</code> . 순서 문턱→VIP 고정	L1	Entity
INV-4	모든 입력에서 <code>0 ≤ final_total ≤ subtotal</code>	L0	Entity
E-1	<code>items is None → TypeError</code>	L0	Boundary*
E-2	<code>price 또는 qty가 음수 → ValueError</code>	L0	Boundary*

docs/PRD.md = SSOT(단일 진실). 계약표 + AC(G-W-T) + 구현 금지(OOS: 쿠폰·세금·배송비·포인트)를 포함한다. 아직 “구현 완료”라 쓰지 않는다 — 현재는 RED 준비.

07

실습 시나리오 — 장바구니 할인 계산기

Hands-on Exercise. “장바구니 할인 계산기”를 TDD로 완성한다. Logic 트랙(subtotal→문턱 할인→VIP·pytest) + UI 트랙(주문 폼→최종 금액·Flask).

① 요구사항 명세 — 그리고 변하지 않는 진실 계약명세

모든 RED 테스트는 이 표에서 도출한다. 표에 없는 동작은 구현하지 않는다(과잉 구현 금지).

명세 ID	범주	명세(단언)	예시(입력 → 기대)	ECB
INV-1	불변식	최종 금액은 음수가 아니다	<code>final_total(...) ≥ 0</code>	Control
INV-2	불변식	할인 후 금액이 할인 전을 넘지 않는다	<code>discount(a) ≤ a</code>	Control
INV-3	불변식	금액 반환값은 원 단위 정수(반올림)	<code>round → int</code>	Control
E-1	예외·경계	빈 장바구니의 합계는 0	<code>[] → 0</code>	Control·Entity
E-2	예외·경계	문턱 미만은 할인 없음 (< 50,000)	<code>49999 → 49999</code>	Control
E-3	예외·경계	문턱 이상은 10% 할인 (≥ 50,000, 경계 포함)	<code>50000 → 45000</code>	Control
AC-1	인수 기준	합계 = $\Sigma(\text{price} \times \text{qty})$	<code>[1000×3, 2000×2] → 7000</code>	Control·Entity
AC-2	인수 기준	문턱 할인 10% (반올림)	<code>60000 → 54000</code>	Control
AC-3	인수 기준	비VIP 최종 = 문턱 할인 결과	<code>[60000×1], vip=F → 54000</code>	Control
U-1	사용 시나리오	VIP는 문턱 할인 후 5% 추가 할인	<code>[60000×1], vip=T → 51300</code>	Control
U-2	사용 시나리오	주문 품이 VIP 최종 금액을 화면에 표시	<code>GET /?vip=1 → "51,300"</code>	Boundary

① 맘 테스트 — 네 개의 렌즈로 명세 뽑기: ① 항상 참이어야 하는 것 → INV(불변식) ② 경계·잘못된 입력 → E(예외·경계) ③ 정상 흐름의 합격 조건 → AC(인수 기준) ④ 사용자는 실제로 어떻게 쓰나 → U(사용 시나리오).

② 테스트 플랜 — 요구사항(계약)에서 TC 뽑기

요구사항(계약)을 곧바로 코드로 옮기지 않고, 먼저 “무엇을 어떻게 검증할지”를 TC로 분해한다. 각 행 = RED 테스트 1개.

TC	검증 의도	입력	기대	경계·비고
TC-1	price×qty 합계 반환	[1000×3, 2000×2]	7000	기본 케이스
TC-2	50,000 이상 10% 할인	50000	45000	경계값 ≥50000
TC-3	50,000 미만 그대로	49999	49999	경계값, 미적용
TC-4	문턱할인 후 VIP 5%	[60000×1], VIP	51300	60000→54000→51300

경계값의 힘 — R2를 TC-2·TC-3 두 개로 나눠, 혼한 “초과(>) vs 이상(≥)” 실수를 RED 단계에서 미리 잡는다.

③ 한 사이클 돌려보기 — subtotal (Python)

1

프롬프트

“테스트만” 작성하게 지시 (src/ 금지)

2

실패 로그

ImportError: cannot import 'subtotal'

3

최소 구현

src/cart.py 에 딱 필요한 함수만

4

통과 확인

1 passed → test·feat 커밋 분리

RED · test_cart.py

```
def test_subtotal_sums_price_times_qty():
    items = [
        {"price":1000,"qty":3},
        {"price":2000,"qty":2},
    ]
    assert subtotal(items) == 7000
```

GREEN · src/cart.py

```
def subtotal(items):
    return sum(
        it["price"] * it["qty"]
        for it in items)
```

④ UI 트랙 — 같은 리듬을 화면에 (Flask)

TC	검증 의도	기대
TC-U1	GET "/" 주문 폼 렌더링	200 · qty·vip 포함
TC-U2	GET "/?vip=1" 금액 표시	"51,300" 포함

재사용 — 계산 로직은 새로 만들지 않는다

Logic 트랙의 `final_total`을 그대로 쓴다. "51,300"은 Logic TC-4와 같은 결과 — 두 트랙이 같은 계산을 공유함을 확인하는 연결 고리다.

두 겹의 검증

```
# src/app.py (요약)
@app.route('/')
def index():
    is_vip = request.args.get('vip') == '1'
    total = final_total(ITEMS, is_vip=is_vip)
    return render_template_string(
        TEMPLATE, total=f'{total:,}')

# test_client() → 응답 HTML 빠른 검사
# Simple Browser / 브라우저 MCP → 시각 검증
```

⑤ 커밋 분리 · Review Mode

git log --oneline

```
feat(ui): 주문 폼 최소 구현 [GREEN]
test(ui): 폼/VIP 실패 테스트 [RED]
feat(logic): final_total [GREEN]
test(logic): final_total [RED]
feat(logic): 문턱 할인 [GREEN]
test(logic): 문턱 할인 [RED]
```

코드를 고치지 않고 GEMINI.md 규칙으로만 점검한다(Plan/Review 모드).

Review Mode 점검 항목 (/review)

- | | | |
|---|--------|---------------------------------|
| ✓ | 우회 | assert True · skip · 예외 삼키기 없는가 |
| ✓ | 과잉 구현 | 요구사항에 없는 코드가 없는가 |
| ✓ | 커밋 분리 | RED와 GREEN이 한 커밋에 안 섞였나 |
| ✓ | 테스트 누락 | 경계값·예외 케이스가 빠지지 않았나 |

08

Gilded Rose — 레거시 코드 QA 프롬프팅

Practical Exercise. Gemini CLI + VS Code QA 프롬프트 라이브러리. 0단계(준비) + 1~9단계(QA) + 보너스. Python 3 + pytest + pytest-cov.

실습 개요 & 아이템 규칙 (Python · Gemini CLI)

📋 실습 개요

- 프로젝트: 기존 Gilded Rose 저장소 사용(폴더 복사)
- 브랜치: main에서 prompting 브랜치 신규 생성
- 기술 스택: Python 3 + pytest + pytest-cov
- 규칙 파일: GEMINI.md (프로젝트 루트, 자동 로딩)
- git switch -c feature/qa-prompting # QA 프롬프팅 작업 브랜치

절대 규칙 (GEMINI.md)

- Item 클래스는 절대 수정 금지
- quality 범위 0~50 (Sulfuras 제외)
- sellIn 음수 허용(기한 초과 상태)
- 테스트 Green 상태에서만 리팩토링 진행

일반 아이템	매일 quality -1 기한 후 -2 최솟값 0
Aged Brie	오래될수록 quality ↑ 기한 후 +2 최댓값 50
Backstage Pass	sellIn<11 → +2, <6 → +3 기한 후 quality=0
Sulfuras	quality 항상 80 고정 sellIn 변화 없음
Conjured ★	매일 -2 기한 후 -4 (일반의 2배) ← 신규 기능

9단계 QA 프로세스 개요 (+0단계·보너스)

단계	제목	주요 산출물
0	실습 준비 (브랜치·환경·GEMINI.md)	브랜치 · GEMINI.md
1	요구사항 분석 (Requirements Analysis)	requirements_analysis.md
2	코드 품질 분석 (SOLID & Code Smell)	code_quality_report.md
3	테스트 계획 (Test Plan)	test_plan.md
4	테스트 케이스 작성 (Test Cases)	테스트 코드 파일
5	테스트 실행 & 결함 분석	defect_list.md
6	Golden Master 자동화 (회귀 안전장치)	Golden Master 테스트
7	리팩토링 지원 (Green 유지)	단계별 체크리스트
8	결함 관리 & 보고	defect_report.md
9	QA 종합 검토 (최종 보고서)	qa_final_report.md
보너스	Conjured 기능 추가 통합 구현	Conjured 구현 + Green

핵심 프롬프트 구조 P-C-T-F (Python · Gemini CLI)

모든 프롬프트에 Persona · Context · Task · Format 4요소와 공통 제약(Item 수정 금지·경계값 포함)을 넣는다.



Persona

시니어 Python QA / 아키텍트 관점



Context

@gilded_rose.py · Python 3, pytest,
pytest-cov



Task

단일 작업 — 분석·테스트·리팩토링 중
하나



Format

Markdown 표 / 저장 파일명 명시

@gilded_rose.py

[P] 시니어 Python 아키텍트 + 클린코드 전문가입니다.

[T] update_quality() 를 SOLID/Code Smell 관점에서 분석해줘.

- SRP/OCF 위반 지점과 근거
- Code Smell: Long Method, Magic Number, Duplicated Code
- Python 특화: 타입 힌트 부재, 문자열 하드코딩, isinstance 미활용
- 리팩토링 우선순위(1~5)와 이유

[F] Markdown 표. code_quality_report.md 로 저장

결함 심각도 분류 (Defect Severity Matrix)

심각도	Python 예시	설명
Critical	Sulfuras quality 변경됨	즉시 수정 필요. 핵심 비즈니스 로직 파괴
Major	Backstage Pass 구간 경계 오류	기능 오동작. 다음 Sprint 내 수정
Minor	sellIn 0일 처리 순서 오류	부분 오동작. 리팩토링 시 함께 수정
Info	불필요한 import·네이밍 불일치	품질 개선 권고. 다음 리팩토링 시 반영

💡 테스트 실패 시 Gemini CLI에 실패 로그(pytest 출력)를 Context로 붙여넣으면 스택트레이스를 읽고 원인·위치(파일:줄)·심각도·최소 수정안을 바로 도출한다.

Conjured 기능 추가 — Item 수정 없이 (Python)

핵심 시험 과제: Item 클래스 수정 없이 Conjured 규칙(매일 -2, 기한 후 -4, 최소 0)을 추가하고 테스트 Green을 확인한다.

Composer 통합 프롬프트

```
@gilded_rose.py @test_gilded_rose.py
@GildedRoseRequirements.txt
```

Conjured 아이템 기능을 추가해줘.

- 규칙: 매일 quality -2,
기한(sell_in < 0) 후 -4, 최소 0
- Item 클래스 수정 없이 구현
- 구현 후 pytest -v 실행해 Green 확인
- pytest --cov 로 커버리지 확인

생성 결과 예시 (Python)

```
def _update_conjured(item):
    rate = 2 if item.sell_in > 0 else 4
    item.quality = max(0, item.quality - rate)

# update_quality() 에서 호출:
# if 'Conjured' in item.name:
#     _update_conjured(item)
```

핵심 — Item 클래스를 수정할 수 없으므로 이름 문자열 기반 분기로 처리한다. 프롬프트에 “Item 클래스 수정 없이” 조건을 반드시 명시한다.

CLOSING

“이 코드는 어떤 계약을 지키고 있는가?”

이 질문에 즉시 답할 수 있게 하는 것 — 그것이 C2C다. 개념 → 명세 → 테스트 → 코드. 네 단계를 ID로 잇는 순간, '통과'는 '증명'이 된다.

1 계약 ID 없는 동작은 구현하지 않는다

2 실패 로그가 최고의 컨텍스트다

3 테스트를 무력화하는 변경은 거절한다

4 GREEN 다음엔 REFACTOR를 둔다

5 구조 변경과 동작 변경을 섞지 않는다

6 C2C 추적성 매트릭스의 빈칸이 다음 질문이다